

[Master PRD & Spec] 다중 플랫폼 알짜차량 자동 탐색기 (AuctionCarRadar)

이 문서는 **Cursor, Claude Code, Windsurf** 등 AI 기반 바이브 코딩(Vibe Coding) 환경에서 프롬프트로 주입하여 대법원 경매, 온비드 공매, 경매마당의 자동차 검색을 자동화하고 정밀 필터링 시스템을 원샷 또는 단계별로 구현하기 위한 마스터 명세서입니다.

1. 프로젝트 개요 및 목적 (Project Overview)

- 프로젝트명: AuctionCarRadar
- 목적: 대한민국 대법원 법원경매정보, 캠코 온비드, 경매마당 3개 주요 경매/공매 플랫폼을 주기적으로 수집하여, 연식, 주행거리, 차키 보유 여부, 사고 이력이 매우 우수한 'A급 알짜친환경 중고차 매물'을 자동으로 필터링하고 실시간 텔레그램 알림을 발송함.
- 핵심 전략:
 - 이미지 OCR을 지양하고, 웹페이지 내 텍스트 요약/상세정보 및 감정평가서 PDF 내 텍스트 키워드 파싱만을 활용하여 속도와 정확도를 극대화함.
 - 멀티 소스(대법원 경매, 온비드 공매, 경매마당)의 매물 정보를 단일 표준 규격(Standard Schema)으로 통합하여 정밀 필터링 및 중복 알림을 관리함.

2. 타겟 사이트 및 수집 데이터 사양 (Target Site & Data Schema)

2.1 타겟 사이트 정보

플랫폼명	메인 URL / 대상 페이지	수집 방식 및 주요 특징
대법원 법원경매	https://www.courtauction.go.kr (경매물건 > 자동차·중기검색)	Playwright 기반 Web Crawling, 감정평가요항표 및 감정평가서 PDF 추출
온비드 (Onbid)	https://www.onbid.co.kr (물건 > 조건검색 > 차량)	Playwright / HTTP API 기반, 공매물건 상세정보 및 공고문/감정평가서 파싱
경매마당	https://madangs.com/search/car (차량검색)	Web Scraping / API Interception, 민간 경매/공매 데이터 및 기일정보 추출

- 크롤링 주기: 매일 2회 (오전 08:00, 오후 18:00)

2.2 수집 데이터 표준 사양 (Unified Data Schema)

구분	수집 항목	비고 / 조건
기본 정보	출처 플랫폼 (court / onbid / madang), 물건/사건번호, 담당기관/법원, 물건명	기본 식별자 및 중복 체크 키
가격 정보	감정평가액, 최저매각가격, 매각기일/입찰마감일	감정가 대비 최저가 비율(유찰률) 자동 계산
차량 스펙	차종/모델명, 연식(제작년식), 주행거리(km), 사용연료	핵심 primary 필터링 항목
상태 키워드	차키 보유 여부 (키 보관, 열쇠, 스페어키, 시동 등)	상세페이지 또는 PDF/감정서 텍스트 검사
위험 키워드	전손, 침수, 대파, 운행불가, 유치권 등	불량 매물 제외용 금지 키워드

3. 타겟 필터링 로직 (Target Filtering Engine)

다음 조건들을 100% 동시 만족(AND 조건)하는 물건만 최종 알림 대상으로 선별함.
(config.yaml로 유연하게 제어)

1. 연식: **2022**년 이후 (2022년식 이상)
2. 주행거리: **50,000 km** 이하 (감정평가요항표, 온비드 상세, 경매마당 스펙 텍스트 파싱)
3. 사용 연료: 하이브리드 또는 전기 (가솔린+전기, 디젤+전기, 수소전기 포함)
4. 차키 보유 유무: 감정평가서 PDF 또는 상세정보 텍스트 내 아래 키워드 중 1개 이상 존재
 - 키 보관, 열쇠 보관, 스페어키, 시동 확인, 계기판 점등
5. 사고/손상 위험 부재: 텍스트 내 아래 금지 키워드가 없을 것
 - 전손, 침수, 불상, 운행 불가, 대파, 유치권, 폐차

4. 시스템 아키텍처 및 디렉토리 구조 (Architecture & Structure)

4.1 파이프라인 흐름

[대법원 / 온비드 / 경매마당 멀티 크롤러 작동]



[1차 수집 & 데이터 표준화 (Standard Dict 변환)]



[1차 정밀 필터링 (연식: 2022년↑ / 주행거리: 50,000km↓ / 연료: 하이브리드·전기)]

▼ (통과 매물 대상)

[2차 상세 파싱 (상세페이지 & 감정평가서 PDF 키워드 분석)]

▼ (키 보유 확인 & 위험 키워드 부재)

[SQLite DB 히스토리 비교 (플랫폼별 Unique ID 기반 중복 방지)]



[텔레그램 Bot 알림 발송]

4.2 권장 프로젝트 디렉토리 구조

AuctionCarRadar/

```
|— config.yaml      # 검색 조건, Telegram API 토큰, 플랫폼별 설정
|— main.py          # 전체 수집 및 필터링 파이프라인 오케스트레이터
|— db_manager.py    # SQLite 기반 중복 알림 방지 및 히스토리 관리
|— scrapers/        # 플랫폼별 독립 수집 모듈
|   |— base_scraper.py # 스크래퍼 공통 인터페이스 / 베이스 클래스
|   |— court_scraper.py # 대한민국 대법원 경매 수집기
|   |— onbid_scraper.py # 캠코 온비드 공매 수집기
|   |— madang_scraper.py # 경매마당 차량검색 수집기
|— modules/
|   |— parser.py      # 정규식 기반 주행거리/연식 파서 및 PDF 파서
|   |— filter.py      # 필터링 규칙 검증 엔진 (2022년↑, 5만km↓, 하이브리드/전기)
|   |— notifier.py    # 텔레그램 메시지 포매팅 및 발송
|— data/
|   |— pdfs/          # 다운로드 임시 파일 저장소
|   |— history.db      # 수집/알림 히스토리 DB
|— requirements.txt
|— README.md
```

5. 모듈별 구현 세부 지침 (Implementation Guide)

모듈 1: config.yaml

search_conditions:

min_year: 2022 # 2022년 이후 (2022년식 이상)

max_mileage: 50000 # 최대 50,000 km

fuel_types: # 희망 연료

- "하이브리드"
- "전기"
- "가솔린+전기"
- "수소전기"

keywords_must_have:

- "키 보관"
- "열쇠"
- "스페어키"
- "시동"
- "점등"

keywords_exclude:

- "전손"
- "침수"
- "운행불가"
- "불상"
- "유치권"
- "폐차"

telegram:

bot_token: "YOUR_TELEGRAM_BOT_TOKEN"

chat_id: "YOUR_TELEGRAM_CHAT_ID"

crawler:

user_agent: "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36"

request_delay_seconds: 3

모듈 2: modules/parser.py (텍스트 & PDF 정밀 파서)

- 연식/주행거리/연료 파싱 정규표현식 예시:

```
import re

def extract_car_details(text: str) -> dict:
    # 연식 추출 (예: "2024년식", "2023년")
    year_match = re.search(r'(20\d{2})\s*년', text)
    year = int(year_match.group(1)) if year_match else None

    # 주행거리 추출 (예: "47,580km", "주행거리 12,345km")
    mileage_match = re.search(r'(?:(주행거리|계기판)[^\d]*(\d{1,7})\s*km', text,
re.IGNORECASE)
    if not mileage_match:
        mileage_match = re.search(r'(\d{1,7})\s*km', text, re.IGNORECASE)

    mileage = int(mileage_match.group(1).replace(',', '')) if mileage_match else 999999

    # 연료 추출
    fuel = "기타"
    if any(term in text for term in ["하이브리드", "Hybrid"]):
        fuel = "하이브리드"
    elif any(term in text for term in ["전기", "EV", "수소"]):
        fuel = "전기"
    elif "휘발유" in text or "가솔린" in text:
        fuel = "휘발유"
    elif "디젤" in text or "경유" in text:
        fuel = "디젤"

    return {
        "year": year,
        "mileage": mileage,
        "fuel": fuel
    }
```

- **PDF** 키워드 파싱:

```
import pdfplumber
```

```
def check_pdf_keywords(pdf_path: str, must_have: list, exclude: list) -> dict:
    full_text = ""
    with pdfplumber.open(pdf_path) as pdf:
        for page in pdf.pages:
            full_text += page.extract_text() or ""

    has_key = any(word in full_text for word in must_have)
    has_danger = any(word in full_text for word in exclude)
    return {"has_key": has_key, "has_danger": has_danger, "raw_text": full_text}
```

모듈 3: modules/notifier.py (텔레그램 알림)

- 통합 메시지 템플릿:

 [A급 알짜 경매/공매 차량 발견!]

 출처: {platform_name} ({court_or_dept})

 사건/물건번호: {case_no}

 차종: {car_name}

 연식: {year}년식 |  주행거리: {mileage:,} km

 연료: {fuel_type}

 감정평가액: {appraisal_price:,} 원

▼ 최저매각가: {min_price:,} 원 ({discount_rate}% 유찰)

 차키 상태: {key_status}

 매각/입찰기일: {auction_date}

 [상세 매물페이지 바로가기]({detail_url})

6. 바이브 코딩용 에이전트 프롬프트 (Vibe Coding Prompts)

AI 도구(Cursor/Claude Code 등)에 단계별로 복사하여 사용할 수 있는 맞춤형 프롬프트입니다.

🎯 Step 1: 프로젝트 구조 및 환경 설정 프롬프트

"첨부된 auction_car_finder_master_spec.md 명세서를 기반으로 Python 프로젝트 기본 디렉토리 구조와 requirements.txt, config.yaml 파일 생성을 도와줘. config.yaml에는 연식 2022년 이상, 주행거리 50,000km 이하, 하이브리드 및 전기 연료 조건이 포함되어야 해. requirements.txt에는 playwright, pdfplumber, pyyaml, requests, python-telegram-bot, beautifulsoup4가 포함되어야 해."

🎯 Step 2: 수집기(Scrapers) 클래스 및 인터페이스 프롬프트

"scrapers/ 디렉토리에 다중 플랫폼 수집기를 작성해줘:

1. base_scraper.py: 공통 스크래퍼 인터페이스 정의 (추상 클래스 BaseScraper).
 2. court_scraper.py: 대한민국 대법원 경매 수집기 (Playwright 기반).
 3. onbid_scraper.py: 온비드(<https://www.onbid.co.kr/>) 자동차 공매 검색 수집기.
 4. madang_scraper.py: 경매마당(<https://madangs.com/search/car>) 차량검색 수집기.
- 각 수집기는 조회 결과를 공통 스펙 Dict(플랫폼, 사건번호, 차종, 감정가, 최저가, 연식, 주행거리, 연료, 상세 URL 등)으로 리턴해야 해."

🎯 Step 3: 정밀 파서 및 필터링 엔진 프롬프트

"modules/parser.py와 modules/filter.py를 완성해줘.

1. parser.py: 텍스트 및 감정평가서 PDF에서 연식, 주행거리(km 정규식), 연료(하이브리드/전기 구분)를 정확하게 파싱하는 함수 작성.
2. filter.py: config.yaml의 조건을 바탕으로 연식 ≥ 2022 , 주행거리 ≤ 50000 , 연료 in [하이브리드, 전기], 필수 키워드(키보관 등) 포함, 금지 키워드(전손 등) 미포함 조건을 100% 만족하는지 검증하는 bool 함수 작성."

🎯 Step 4: 통합 오케스트레이터 및 알림 연동 프롬프트

"main.py와 db_manager.py를 구현해줘. SQLite DB를 사용해 플랫폼명+사건번호 조합으로 중복 발송을 방지하고, 3개 플랫폼(대법원, 온비드, 경매마당)에서 조건에 맞는 알짜 매물을 수집 후 텔레그램으로 전송하는 전체 실행 스크립트를 작성해줘."

7. 예외 처리 및 크롤링 방어 대책 (Robustness & Security)

1. 플랫폼별 세션 및 재시도: 온비드 및 대법원 사이트는 세션 타임아웃이 발생할 수 있으므로 **tenacity** 라이브러리를 활용해 실패 시 최대 3회 재시도(**@retry**)를 적용함.
2. 차단 방지 (**Rate Limiting**): 각 요청 및 페이지 이동 시 **time.sleep(random.uniform(2.5, 4.5))** 초를 적용하여 인간의 탐색 패턴을 모사함.
3. 온비드/경매마당 동적 페이지 처리: **JavaScript** 렌더링 또는 **API** 호출 방식을 사용하는 페이지 구조에 대응하여 **Playwright wait_for_selector** 또는 네트워크 응답 가로채기(**Interception**) 활용.
4. 디스크 용량 관리: 분석이 완료된 임시 **PDF** 파일은 분석 완료 직후 **os.remove()**로 삭제 처리.

8. 최종 검증 체크리스트 (Testing Checklist)

- ☐ **config.yaml**의 조건이 연식 2022년 이상, 주행거리 50,000km 이하, 연료 하이브리드/전기로 세팅되었는가?
- ☐ 대법원, 온비드, 경매마당 3개 플랫폼 수집기 모듈이 동일한 사양의 표준 **Dict** 형태로 데이터를 반환하는가?
- ☐ 주행거리 파서가 47,580km, 50000 km, 12,345 킬로미터 등 다양한 표기법을 숫자로 정상 변환하는가?
- ☐ 온비드/대법원 감정평가서 **PDF** 분석 시 '키 보관' 문구 탐지 및 '전손/유치권' 금지 키워드 필터가 올바르게 작동하는가?
- ☐ **DB**에 등록된 중복 사건번호/물건번호가 두 번째 스크랩 시 알림 대상에서 정상 제외되는가?